Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**MCA**	
Course	**Programming from First Principles**	
Topic Name	**Definitions: a means of looping indefinitely**	
Topic Code	**-**	
Unit/Module No. & Name	**12**	**Recursion**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. This self-learning material explains recursion as a method of solving problems in programming.
2. It introduces the meaning of recursion, the structure of a recursive function, practical examples such as factorial, summation, and Fibonacci, and the comparison between recursion and iteration.
3. The topic is important because recursion helps break a large problem into smaller subproblems until a simple Stopping condition is reached.
4. By the end of this lesson, students should be able to define recursion, identify base case and recursive case, trace recursive execution, and compare recursion with iteration in Python programming.

1.0 INTRODUCTION

Recursion is an important mathematical and programming concept. In programming, recursion is used when a problem can be divided into smaller versions of the same problem. Instead of solving the whole task at once, the function keeps calling itself with a changed parameter until it reaches a stopping point. This makes recursion a powerful method for many problems in mathematics, data structures, and algorithms. The topic also helps students understand how functions work internally, especially how repeated function calls are stored and executed. Because of this, recursion is both a practical and a conceptual topic in programming.

Recursion is worth studying because many important problems are naturally recursive. Examples include factorial, Fibonacci numbers, binary tree traversal, graph algorithms, linked lists, binary search, merge sort, quick sort, and heap sort. In these problems, the same pattern appears again and again, so recursion provides a direct and elegant solution. It also helps students think in terms of problem decomposition. Instead of writing many loops and control steps, they learn how to describe a problem in terms of itself. This builds deeper programming understanding and prepares students for more advanced computer science concepts and algorithm design.

2.0 MEANING AND DEFINITION

2.1 Meaning Of Recursion

Recursion is a programming technique in which a function solves a problem by calling itself again and again with a modified value. Each new call handles a smaller version of the original problem. This process continues until the problem becomes so small that it can be solved directly. In simple words, recursion means defining a solution in terms of itself. It is widely used in mathematics and programming because many patterns naturally repeat in smaller forms. A recursive solution is often shorter and cleaner than an iterative one, but it must be written carefully to avoid endless function calls.

2.1.1 Formal Definition Of Recursion

In academic terms, recursion is a mathematical and programming concept in which a function directly or indirectly calls itself to solve a problem by reducing it into smaller subproblems. A recursive function usually contains a terminating condition and a self-referential step. The terminating condition ensures that the repeated calls stop at the correct time, while the self-referential step ensures that the problem continues to move toward the solution. Thus, recursion is not only repeated calling. It is a controlled and structured way of solving a large problem through smaller, similar problem instances.

3.0 STRUCTURE OF A RECURSIVE FUNCTION

3.1 Base Case

The base case is the stopping condition of a recursive function. It tells the function when to stop making further recursive calls and return a direct answer. Without a base case, the function would continue calling itself forever. This may cause a crash or memory exhaustion because the system keeps storing new calls on the stack. Therefore, the base case is necessary for safe and correct recursion. In practical terms, the base case usually handles the smallest possible input, such as factorial of 1 or summation of 0, where the answer is already known directly.

3.1.1 Role Of Base Case In Termination

The base case plays the role of termination in recursion. It acts like the final checkpoint that stops the repeated self-calling process. Each recursive call must move closer to this base case. If the program reaches the base case, it starts returning results back through previous function calls. If it never reaches the base case, recursion becomes infinite. In Python, too many recursive calls can lead to stack overflow or recursion depth error. Thus, the base case is not only a logical condition. It is also a practical safety feature that protects the program from running endlessly.

3.2 Recursive Case

The recursive case is the part of the function where the problem is reduced into a smaller subproblem and the same function is called again. This part carries the main logic of recursion. Instead of solving everything at once, the recursive case handles one part of the task and sends the rest to another call of the same function. The new call always uses a modified parameter, usually smaller than the current one. This is important because the function must move toward the base case. Without this change, the recursion would repeat the same state forever.

3.2.1 Modified Parameter In Recursive Calls

A recursive function must call itself with a modified parameter. This change is what pushes the problem toward the base case. For example, factorial of n uses factorial of n minus 1, and summation of n uses summation of n minus 1. The value is reduced in each step, so the function

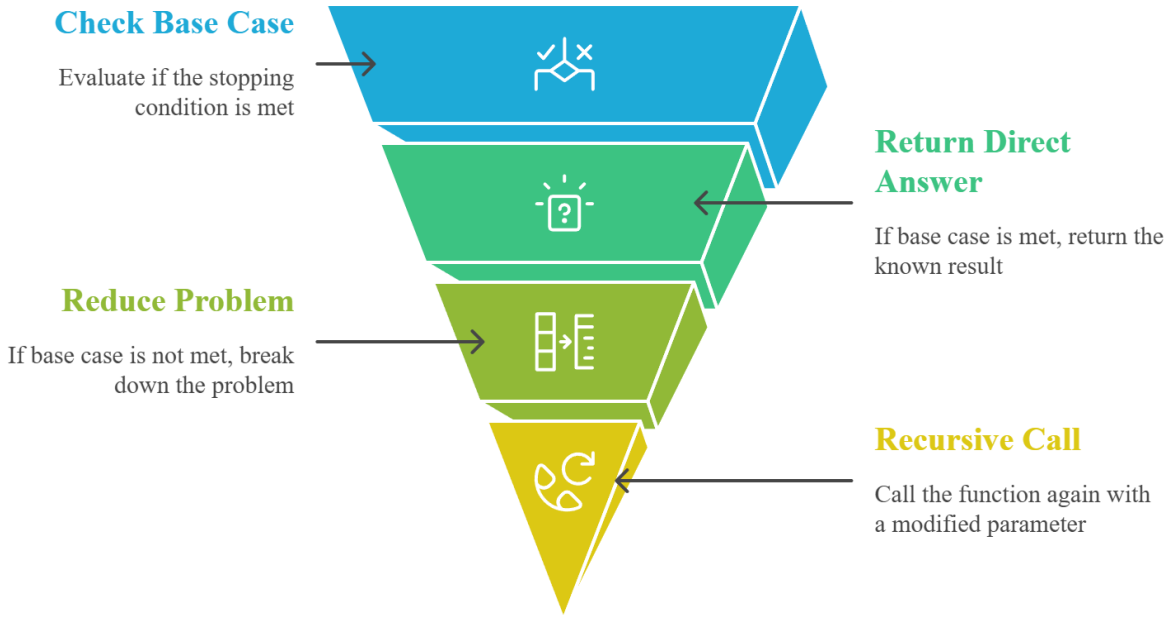
eventually reaches the stopping condition. If the parameter does not change properly, the recursion will not progress and will never terminate. Therefore, the modified parameter is one of the most important parts of recursive design. It ensures both correctness and completion of the recursive solution.

4.0 FACTORIAL USING RECURSION

4.1 Mathematical Idea Of Factorial

Factorial is one of the most common examples used to explain recursion. The factorial of a positive integer n is the product of all integers from 1 up to n. In recursive form, factorial of n is defined as 1 when n is equal to 1. Otherwise, it is n multiplied by factorial of n minus 1. This makes it a natural recursive problem because the factorial of a number depends on the factorial of a smaller number. The formula is simple, repetitive, and clearly shows both the base case and the recursive case.

Fig. 1 Recursive Function Execution Flow



This flowchart illustrates recursion steps: checking the base case, returning results if met, reducing the problem otherwise, and making recursive calls until the solution is achieved.

4.1.1 Execution Of Factorial Recursion

Suppose we find factorial of 3. Since 3 is not the base case, the function returns 3 multiplied by factorial of 2. Then factorial of 2 returns 2 multiplied by factorial of 1. When factorial of 1 is reached, the base case returns 1. After that, the results move backward. Factorial of 2 becomes 2

multiplied by 1, which is 2. Then factorial of 3 becomes 3 multiplied by 2, which is 6. This backward return process shows how recursive calls build up and then resolve step by step through the call chain.

4.2 Python Structure Of Factorial

In Python, the recursive factorial function is written like a normal function using def. Inside the function, the first part checks the base case. If n is equal to 1, it returns 1. Otherwise, it returns n multiplied by factorial of n minus 1. After defining the function, the program can take input from the user, pass it as an argument, store the returned answer in a variable, and display it. This example is useful because it shows both the recursive logic and the basic programming structure of input, processing, and output.

5.0 SUMMATION USING RECURSION

5.1 Mathematical Idea Of Summation

Summation is another simple example used to explain recursion. The sum of n means adding all whole numbers from n down to 1. In recursive form, if n is equal to 0, the answer is 0. Otherwise, the answer is n plus sum of n minus 1. This structure is very similar to factorial, except multiplication is replaced with addition. It is a good example because it helps students see that the recursive pattern can be used for different operations. The key idea remains the same: solve one part now and solve the smaller remainder recursively.

5.1.1 Execution Of Summation Recursion

Suppose we find sum of 4. Since 4 is not the base case, the function returns 4 plus sum of 3. Then sum of 3 returns 3 plus sum of 2. Sum of 2 returns 2 plus sum of 1, and sum of 1 returns 1 plus sum of 0. At this point, the base case is reached and returns 0. The values then return backward: 1 plus 0 gives 1, 2 plus 1 gives 3, 3 plus 3 gives 6, and 4 plus 6 gives 10. Thus, sum of 4 is 10.

5.2 Python Structure Of Summation

In Python, the summation function uses the same recursive design pattern as factorial. The function first checks whether n is 0. If yes, it returns 0. Otherwise, it returns n plus the result of calling the same function with n minus 1. This makes the program easy to read because the mathematical idea and the code structure are closely connected. The summation example is useful for students because it shows that recursion is not limited to multiplication-based problems. It can also solve addition-based problems clearly and effectively.

6.0 FIBONACCI USING RECURSION

6.1 Mathematical Idea Of Fibonacci

The Fibonacci example is different from factorial and summation because it creates two recursive calls instead of one. In this function, if n is 0 or 1, the result is 1. Otherwise, the answer is Fibonacci of n minus 1 plus Fibonacci of n minus 2. This means each call can branch into two more calls, making the structure hierarchical rather than linear. The Fibonacci example is important because it shows that recursion does not always move in one straight line. It can also create a tree-like pattern of function calls.

6.1.1 Execution Of Fibonacci Recursion

Suppose we find Fibonacci of 3. Since 3 is not a base case, the function returns Fibonacci of 2 plus Fibonacci of 1. Fibonacci of 1 is already a base case, so it returns 1. Fibonacci of 2 is still not a base case, so it returns Fibonacci of 1 plus Fibonacci of 0. Both of these are base cases and return 1 each. Thus, Fibonacci of 2 becomes 2, and Fibonacci of 3 becomes 2 plus 1, which is 3. This example shows branching recursion and how answers combine from multiple smaller calls.

6.2 Python Structure Of Fibonacci

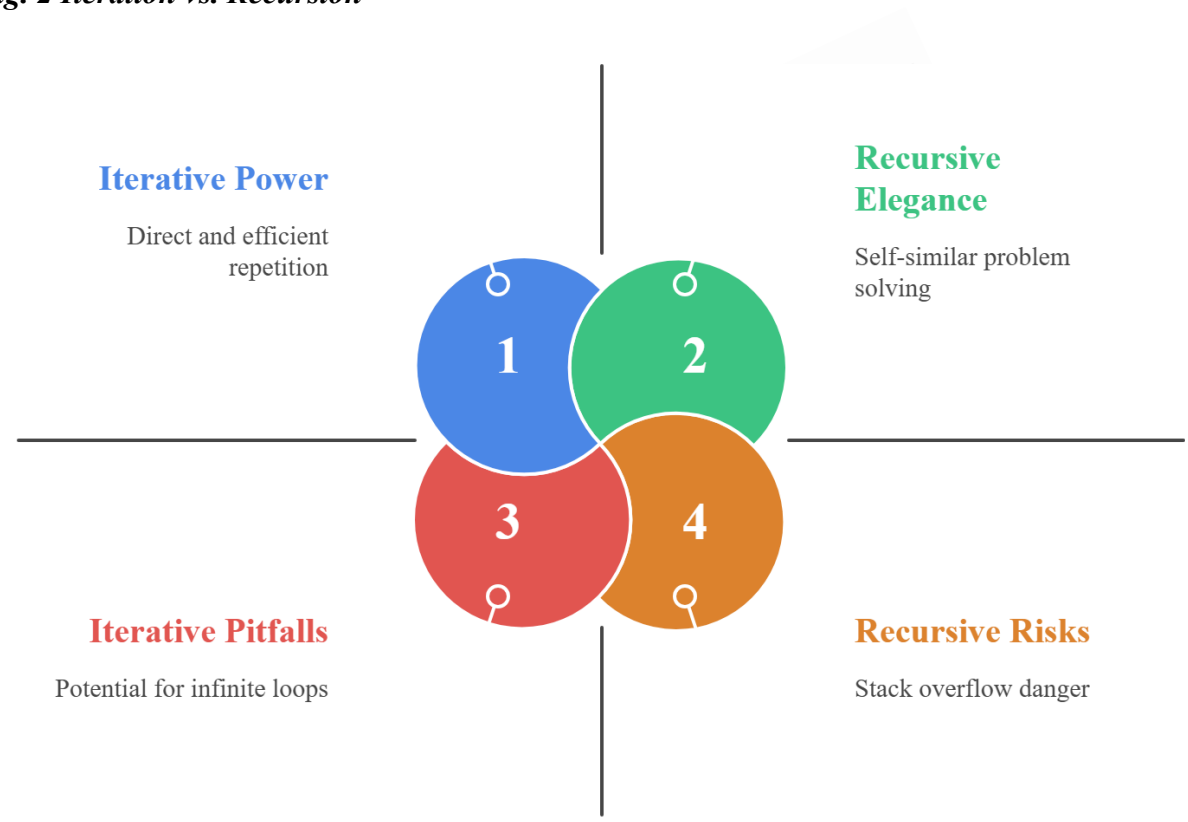
In Python, the Fibonacci function first checks whether n is 0 or 1. If yes, it returns 1. Otherwise, it returns the sum of two recursive calls: Fibonacci of n minus 1 and Fibonacci of n minus 2. This code is short and mathematically clear, but it also shows why recursion can become expensive when multiple calls are made repeatedly. The Fibonacci example helps students understand how recursion may follow a hierarchical pattern and why recursive thinking is powerful but must be used carefully in programming.

7.0 COMPARISON BETWEEN ITERATION AND RECURSION

7.1 Mode Of Implementation

Iteration is implemented using looping statements such as for and while. Recursion is implemented using a function that calls itself. In iteration, repetition is controlled by loop conditions and control variables. In recursion, repetition is controlled by repeated function calls and the base case. Both solve repeated tasks, but they do so in different ways. Iteration often looks more direct for simple repetition, while recursion is more natural for self-similar problems. This difference in implementation style makes each method suitable for different kinds of programming problems.

Fig. 2 Iteration vs. Recursion



This diagram compares iteration and recursion, highlighting iterative efficiency and risks like infinite loops, alongside recursive elegance in solving self-similar problems and potential stack overflow issues.

7.1.1 State And Progression

In iteration, the state is controlled by variables such as loop counters. The program moves toward the ending condition by increasing or decreasing these variables. In recursion, the state is controlled by parameter values stored in the call stack. The program moves toward the base case by changing these parameter values in each recursive call. Thus, both methods progress toward termination, but one does so through loop control variables and the other through modified function arguments and stacked function frames.

7.2 Termination, Size, And Speed

Iteration stops when the loop condition becomes false. Recursion stops when the base case becomes true. Iterative code is often larger, but it usually runs faster because it avoids repeated function call overhead. Recursive code is often shorter and cleaner, but its execution can be slower because every call pushes a new frame onto the stack. This adds extra work during execution. Thus, recursion may be elegant, but iteration is often more efficient in time and memory for many simple tasks.

7.2.1 Effect Of Missing Termination

If iteration is written without a proper termination condition, it leads to an infinite loop that keeps using CPU cycles. If recursion is written without a correct base case or modified parameter, it causes repeated function calls until stack overflow or crash occurs. In Python, recursion depth is limited by default, often around 1000 calls unless changed. This makes proper termination especially important in recursive programming. Therefore, both iteration and recursion need a correct stopping condition, but the practical effect of missing it differs in the two approaches.

1. Recursion is a method in which a function calls itself.
2. A recursive function contains a base case and a recursive case.
3. The base case stops the recursion.
4. The recursive case reduces the problem to a smaller one.
5. Factorial, summation, and Fibonacci are classic examples of recursion.
6. Factorial and summation follow a linear recursive pattern.

7. Fibonacci follows a branching or hierarchical recursive pattern.
8. Iteration uses loops, while recursion uses self-calling functions.
9. Iteration is usually faster, but recursion is often shorter and cleaner.
10. Missing termination causes infinite loops in iteration and stack overflow in recursion.

8.0 KEYWORDS TABLE

Keyword	Touch Vocabulary
Recursion	A function calling itself
Base Case	The stopping condition
Recursive Case	The self-calling part
Modified Parameter	Changed input for the next call
Factorial	Product of numbers from 1 to n
Summation	Addition of numbers from 1 to n
Fibonacci	A recursive number pattern
Call Stack	Memory used for function calls
Iteration	Repetition using loops

Infinite Loop	Endless loop execution
Stack Overflow	Too many recursive calls
Divide And Conquer	Breaking a problem into smaller parts
Binary Search	A recursive search algorithm
Tree Traversal	Visiting nodes in a tree structure
Algorithm Design	Planning problem-solving steps

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited